Confluence2.0

Development Handoff Summary

Unified Knowledge Platform with RAG Search

Document date: June 21, 2026

Application version: 2.0.0

Repository: CJ-Covance/AIProjects

1. Executive Summary

Confluence2.0 is an enterprise knowledge platform that lets users ask natural-language questions and receive consolidated, cited answers from an organizational knowledge base. Content is organized in a four-level hierarchy (Source → Domain → Project → Web Pages), stored on disk and in SQLite, and searched using Retrieval-Augmented Generation (RAG).

The platform was built as a full-stack POC/demo: FastAPI backend, Next.js 14 frontend, OpenAI and Google Gemini integration with automatic provider fallback, folder-based file ingestion (PDF, HTML, MD, TXT), activity logging, and a management UI for CRUD operations.

2. Product Scope & Features Delivered

- Ask — Natural-language search with scope filters (source, domain, project) and inline citations.
- Browse — Hierarchical knowledge explorer with page counts.
- Manage — CRUD for sources, domains, projects, and web pages; sync/upload project files.
- Logs — Activity and error log viewer with filtering.
- RAG pipeline — Chunking, embedding, vector retrieval, and grounded answer generation.
- Folder sync — Auto-import files from knowledge_base/ during search or manual sync.
- Multi-provider AI — OpenAI and Google Gemini with LLM_PROVIDER=auto fallback on quota errors.
- Keyword fallback — Term-overlap retrieval when embeddings fail or scores are low.
- Demo branding — UI rebranded to Confluence2.0 with version footer (v2.0.0).

3. System Architecture

3.1 Knowledge Hierarchy

Source (root) → Domain → Project → Web Pages. Each level can map to a folder under backend/knowledge_base/. Files placed in source root folders (e.g. Cancer/cancer1.txt) are synced to the first project under that source.

3.2 RAG Pipeline

- Sync — Import disk files into WebPage records (folder_sync service).
- Index — Split content into chunks; embed via OpenAI or Google; store in chunks table.
- Retrieve — Cosine similarity + keyword fallback; top-K chunks selected.
- Generate — LLM produces answer strictly from retrieved context with citations.

3.3 Tech Stack

Layer	Technology
Frontend	Next.js 14, React 18, TypeScript, Tailwind CSS
Backend	FastAPI, SQLAlchemy, Pydantic, Uvicorn
Database	SQLite (atlas.db) with embedding vectors stored as JSON
AI — OpenAI	text-embedding-3-small + gpt-4o-mini
AI — Google	gemini-embedding-001 + gemini-2.0-flash
File parsing	pypdf, native HTML/MD/TXT readers

4. Repository Structure

backend/app/main.py — FastAPI entry point, health endpoints

backend/app/models.py — SQLAlchemy models (Source, Domain, Project, WebPage, Chunk, ActivityLog)

backend/app/routers/ — REST API route handlers

backend/app/services/rag.py — Search and answer generation

backend/app/services/llm_provider.py — OpenAI/Google switching and connectivity tests

backend/app/services/folder_sync.py — Disk-to-database file import

backend/app/services/indexer.py — Chunking and embedding pipeline

backend/knowledge_base/ — On-disk knowledge files (gitignored)

backend/.env — API keys and configuration (not in git)

frontend/src/app/ — Next.js pages: /, /browse, /manage, /logs

frontend/src/lib/api.ts — Backend API client

docs/ — Documentation and this handoff PDF

5. Local Development Setup

5.1 Prerequisites

- Python 3.9+ (tested on 3.9.5 Windows and 3.12 Linux)
- Node.js 20+
- OpenAI API key and/or Google Gemini API key

5.2 Backend

```
cd backend
python -m venv .venv
.venv\Scripts\activate (Windows)
python -m pip install -r requirements.txt
copy .env.example .env
python seed_data.py
python -m uvicorn app.main:app --reload --port 8000
```

5.3 Frontend

```
cd frontend
npm install
npm run dev
```

Open http://localhost:3000 (frontend). Backend API: http://localhost:8000

6. Environment Configuration

All backend configuration lives in backend/.env. Cursor IDE API keys are NOT used — the application calls OpenAI and Google APIs directly.

Variable	Default	Description
LLM_PROVIDER	auto	openai | google | auto (fallback on quota errors)
OPENAI_API_KEY	—	OpenAI API key
GOOGLE_API_KEY	—	Google Gemini key (GEMINI_API_KEY alias accepted)
GOOGLE_EMBEDDING_MODEL	gemini-embedding-001	Do not use retired text-embedding-004
GOOGLE_CHAT_MODEL	gemini-2.0-flash	Gemini chat model
KNOWLEDGE_BASE_ROOT	knowledge_base	Root folder for on-disk files
OPENAI_SSL_VERIFY	true	Set false only for dev SSL issues on Windows

6.1 Verify AI Connectivity

- GET http://localhost:8000/api/health — shows key status and env file path
- GET http://localhost:8000/api/health/test-llm — runs live embed + chat tests
- If OpenAI quota is exhausted, set LLM_PROVIDER=google and use a valid GOOGLE_API_KEY

7. Key API Endpoints

Method	Endpoint	Description
GET	/api/health	Health check and provider status
GET	/api/health/test-llm	Test OpenAI/Google connectivity
POST	/api/search	RAG question answering
POST	/api/reindex	Re-index pages in scope
GET	/api/hierarchy	Full knowledge tree
GET/POST/PATCH/DELETE	/api/sources, /domains, /projects, /pages	CRUD operations
POST	/api/projects/{id}/sync-folder	Import files from disk
POST	/api/projects/{id}/upload	Upload file to project folder
GET	/api/logs	Activity and error logs

8. Demo Workflow

- Manage: Create Source (e.g. Cancer) → Domain → Project.
- Place files in backend/knowledge_base/{Source}/ or project subfolders.
- Use Sync folder or Upload file on Manage, or simply Ask a question (auto-sync).
- Ask: Select scope filter, enter question (e.g. Benign vs Malignant Tumors), click Ask.
- Review answer, citations, and search diagnostics (pages/chunks indexed).
- Logs: Check indexing or API errors if search returns no results.

9. Known Issues & Troubleshooting

Issue	Cause	Resolution
OpenAI 429 quota error	Billing/quota exhausted	Set LLM_PROVIDER=google; add GOOGLE_API_KEY; restart backend
Google 404 embedding model	text-embedding-004 retired	Use GOOGLE_EMBEDDING_MODEL=gemini-embedding-001
pip install fails on Python 3.9	google-genai 1.47+ needs Python 3.10	Use pinned google-genai==1.46.0 in requirements.txt
Page saved but indexing failed	API key missing or provider error	Check /api/health/test-llm; fix .env; re-save page
No search results	Missing chunks or wrong scope	Ensure Domain+Project exist; run POST /api/reindex
SSL certificate error (Windows)	Corporate proxy/certs	pip install certifi; or OPENAI_SSL_VERIFY=false (dev only)
Logs page 404	Wrong URL	Use http://localhost:3000/logs (not port 8000)

10. Handoff Checklist

- Clone repository and checkout branch cursor/atlas-knowledge-platform-86a3 (or main after merge).
- Create backend/.env from .env.example with valid GOOGLE_API_KEY or OPENAI_API_KEY.
- Run pip install -r requirements.txt and npm install in frontend.
- Start backend (port 8000) and frontend (port 3000).
- Verify GET /api/health/test-llm returns any_provider_working: true.
- Create knowledge hierarchy in Manage and add/sync content.
- Run demo search queries and confirm citations appear.
- Review PR #4 on GitHub for full change history.

11. Recommended Future Enhancements

- PostgreSQL + pgvector for production-scale vector search
- SSO / OIDC authentication and role-based access control
- Hybrid retrieval (BM25 + semantic) and re-ranking
- Connectors for Confluence, SharePoint, Jira
- Document versioning, audit trails, and approval workflows
- Docker/Kubernetes deployment with secrets management

This document summarizes the development state of Confluence2.0 as of the handoff date. For live API exploration, use the FastAPI docs at http://localhost:8000/docs when the backend is running.